Studio Nx

JS 167 — initRecoverTabs()

2026-10-15

JS 167 — `initRecoverTabs()` (탭 클릭과 키보드 이동)

항목	내용
파일	`studionx/frontend/static/js/core/recover.js` (내부 도우미 `tabNameOf()` 포함)
작업	P1-28 계정 찾기 화면: JS 세 번째 함수
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	166 `selectTab()`
다음	열기 연결: `Can't sign in?`, `home.html#recover`, `Back to Sign In`, 모달을 닫으면 로그인 화면으로 초기화

1. 하는 일

복구 화면의 탭 목록(`role="tablist"`)에 마우스와 키보드 조작을 연결해서, 이전 함수 `selectTab()`을 부르게 합니다.

조작	동작
탭을 누름	그 탭을 고름
→ (탭에 커서가 있을 때)	다음 탭. 끝에서는 처음으로 이어짐
←	이전 탭. 처음에서는 끝으로 이어짐
`Home`, `End`	첫 탭, 마지막 탭

옮기면 **바로 그 탭을 고르고 커서도 따라갑니다**(WAI-ARIA 탭 패턴의 자동 선택). 처리한 키만 기본 동작을 막고, `Ctrl`, `Alt`, `Meta`, `Shift`를 함께 누르면 처리하지 않습니다(브라우저와 화면 낭독기의 단축키를 가로채지 않기 위해서입니다).

2. 코드

```
function tabNameOf(tab) { /* id 가 recover-tab-<이름>이고 이름이 알려진 값이면 이름, 아니면 null */ }

export function initRecoverTabs(modal) {
  if (!modal) return false;
  const list = modal.querySelector('[role="tablist"]');
  if (!list) return false;

  list.onclick = function (event) {
    const tab = event.target && event.target.closest ? event.target.closest('[role="tab"]') : null;
    const name = tabNameOf(tab);
    if (name !== null) selectTab(modal, name);
  };

  list.onkeydown = function (event) {
    if (event.ctrlKey || event.altKey || event.metaKey || event.shiftKey) return;
    const key = event.key;
    if (key !== 'ArrowRight' && key !== 'ArrowLeft' && key !== 'Home' && key !== 'End') return;
    const current = event.target && event.target.closest ? event.target.closest('[role="tab"]') : null;
    if (tabNameOf(current) === null) return;
    /* names = 탭 이름 목록, index = 가려는 위치(→ 는 +1, ← 는 -1, 끝에서 이어짐, Home=0, End=마지막) */
```

```
    event.preventDefault();
    selectTab(modal, names[index], true);
  };
  return true;
}
```

최신 문법으로 쓰면 (공부용)

```
export const initRecoverTabs = (modal) => {
  const list = modal?.querySelector('[role="tablist"]');
  if (!list) return false;

  list.onclick = (e) => {
    const name = tabNameOf(e.target.closest?.('[role="tab"]'));
    if (name) selectTab(modal, name);
  };
  list.onkeydown = (e) => {
    if (e.ctrlKey || e.altKey || e.metaKey || e.shiftKey) return;
    const names = [...list.querySelectorAll('[role="tab"]')].map(tabNameOf).filter(Boolean);
    const at = names.indexOf(tabNameOf(e.target.closest?.('[role="tab"]')));
    const next = {
      ArrowRight: (at + 1) % names.length,
      ArrowLeft: (at - 1 + names.length) % names.length,
      Home: 0,
      End: names.length - 1,
    }[e.key];
    if (at < 0 || next === undefined) return;
    e.preventDefault();
    selectTab(modal, names[next], true);
  };
  return true;
};
```

최신 문법	실제 코드	뜻
`modal?.querySelector(…)`	`if (!modal) return false;` 뒤에 조회	모달이 없으면 건너뜁니다
`e.target.closest?.(…)`	`event.target && event.target.closest ? … : null`	`closest`가 없는 대상도 안전하게 다룹니다
키별 값을 담은 객체 `{ ArrowRight: …, … }[e.key]`	`if / else if` 사슬	키에 따라 가려는 위치를 정합니다
`.map(…).filter(Boolean)`	`forEach`로 `names`에 담기	알려진 탭 이름만 모읍니다

문법 풀이

식	뜻
`(index + 1) % names.length`	나머지 연산으로 끝에서 처음으로 이어지게 합니다(2개면 0→1→0)
`(index - 1 + names.length) % names.length`	처음에서 이전으로 가면 음수가 되므로 길이를 더해 끝으로 이어지게 합니다
`event.target.closest('[role="tab"]')`	눌린 곳(탭 안의 글자 등)에서 가장 가까운 탭 버튼을 찾습니다
`list.onclick = …`	처리기를 대입합니다. 이미 있던 처리기를 바꿔서 두 번 연결해도 하나만 남습니다

3. 왜 이렇게

선택	이유
화살표로 옮기면 바로 선택 (자동 선택)	탭이 둘뿐이고 패널 전환이 가벼워서 "옮기고 Enter로 확정"하는 수동 방식보다 간단합니다
조합 키는 처리하지 않음	`Ctrl+→`는 글자 단위 이동, `Alt+←`는 브라우저 뒤로 가기 같은 기존 단축키입니다. 가로채면 안 됩니다
처리한 키만 `preventDefault()`	화살표가 화면을 스크롤하는 기본 동작은 막되, `a`나 `↑`는 그대로 둡니다
처리기를 **탭 목록 하나에만** 달고 이벤트 전달을 이용	탭마다 따로 달지 않아도 누른 탭을 `event.target`으로 알 수 있고, 탭을 나중에 늘려도 그대로 동작합니다. 입력칸은 탭 목록 밖에 있어서 칸 안의 화살표 키는 처리기에 닿지 않습니다
`onclick`, `onkeydown`에 대입	이 함수를 여러 번 불러도 처리기가 쌓이지 않게 하려는 습관입니다(홈 화면의 다른 함수들과 같음)
탭 이름은 `tabNameOf()`로 한 곳에서	`recover-tab-<이름>` 규칙을 한 곳에서만 해석합니다

4. 시험 (실제 서버, 마우스와 키보드 이벤트, 39개 통과)

헤드리스 Chrome에 **실제 마우스 클릭과 키 입력**(`Input.dispatchMouseEvent`, `Input.dispatchKeyEvent`)을 보냈고 `home.html`, `join.html`에서 같은 시험을 했습니다.

확인	결과
두 번 연결해도 `true`	✓ ✓
Email, Password 탭을 마우스로 누르면 선택되고 패널이 바뀜	✓ ✓ ✓ ✓
`→`: 다음 탭으로, 커서가 따라가고 `tabindex`도 바뀜	✓ ✓
`→`가 끝에서 처음으로, `←`가 처음에서 끝으로 이어짐, `←`로 돌아옴	✓ (6개)
`End`는 마지막 탭, `Home`은 첫 탭, 이미 첫 탭에서 `Home`을 눌러도 그대로	✓ (6개)
다른 키(`a`)는 처리하지 않음	✓ ✓
`Ctrl`, `Alt`, `Meta`, `Shift`와 함께 누른 `→`는 처리하지 않음	✓ (8개)
처리한 키(`→ ← Home End`)만 기본 동작을 막고 `a`, `↑`, `Ctrl+→`는 막지 않음	✓ ✓
입력칸에서 `→`를 눌러도 탭이 바뀌지 않음	✓ ✓
탭 목록의 빈 곳을 눌러도 바뀌지 않음	✓ ✓
모달이 `null`이거나 탭 목록이 없으면 `false`	✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
클릭 처리 제거	2개 실패
끝에서 처음으로 안 이어지게	6개 실패
`Home`과 `End`를 뒤바꿈	16개 실패
옮긴 뒤 커서가 따라가지 않게	10개 실패
`preventDefault()` 제거	2개 실패
조합 키 검사 제거	6개 실패

바꾼 것	결과
`→`를 이전 탭 쪽으로 바꿈	실패 없음
탭 밖에서 온 키도 처리	실패 없음
처리기를 `addEventListener`로 두 번 연결	실패 없음

시험이 잡지 못한 세 가지, 모두 시험의 빈틈이 아니라 지금 구조의 성질입니다.

1. `→`**와** `←`**의 방향:** 탭이 **둘**이면 "다음"과 "이전"이 같은 탭이라 방향을 뒤바꿔도 결과가 같습니다. 탭이 셋 이상이 되면 방향이 드러나는데, 지금은 탭 이름 목록(`RECOVER_TABS`)에 둘뿐이라 만들어 시험할 수 없습니다. 탭을 늘릴 때 시험을 더해야 합니다.
2. **탭 밖의 키:** 처리기가 탭 목록에만 달려 있어서, 입력칸의 키 입력은 처리기에 도달하지 않습니다. 그 검사는 방어용 중복입니다.
3. **처리기 겹침:** 처리기를 두 번 달아도 둘 다 같은 `event.target`으로 **같은 탭을 고르므로** 결과가 같습니다(멱등). `onclick`, `onkeydown` 대입은 겹침을 줄이는 습관이지만 이 함수에서는 동작을 가르지 않습니다. 처음 시험에서 "처리기가 겹치면 한 바퀴 돌아 원위치"라고 가정했다가, 실제로는 같은 탭을 두 번 고르는 것뿐임을 확인하고 시험의 설명을 바로잡았습니다.

5. 아직 이 함수를 부르는 곳이 없습니다

복구 화면을 여는 연결(다음 함수)이 이 함수를 부릅니다. 그 전에는 복구 화면 자체가 열리지 않아서 탭을 쓸 수 없습니다.